

ProductionManagementAI — 製造指示一覧のクエリ — データベース設計書（テーブル定義書）

002_DB — 要件 REQ-020～REQ-027（`work-items/WI-003/brief.md`）、基本設計 002_BD（`docs/en/010_basic-design/002/002_BD_製造指示一覧.md`）。002_DD と 002_DD-API（`docs/en/020_detailed-design/002/`）を実装する。

WI-005 による改訂（2026-09-23）：初期データの備考を日本語にした（初期データの表を参照）。それ以外の変更はない。

この文書は**テーブルも列も追加しない**。画面 B は 001_DB が定義したスキーマを読む。データベースに求めるのは、フィルターと既定のソートを支えるインデックス、定義された件数クエリ、安定したページング順、デモ用データである。001_DB が `products`、`production_orders`、`production_order_number_counters` について定めていることはすべてそのまま有効であり、ここでは繰り返さない。

物理命名は 000_DB と 001_DB に従う：`snake_case`、EF Core の `pk_ / fk_ / ix_ / ck_` 形式の制約名。

文書管理（改版履歴）

項目	値
文書 ID	002_DB
システム名	ProductionManagementAI
作業項目	WI-003
データベース	<code>production_management_ai</code> 、スキーマ <code>public</code>
DBMS	PostgreSQL 17
文字コード	UTF-8
作成者	Claude（ThanhTN の依頼による）
作成日	2026-09-22
最終更新者	Claude（ThanhTN の依頼による）
最終更新日	2026-09-28

No	対象	作業項目	変更内容	変更日	変更者
1	全体	WI-003	新規作成（DB-003として）：一覧クエリのアクセスパターン、インデックス 2 つ、デモ用製造指示 80 件の初期データ	2026-09-22	Claude（ThanhTN の依頼による）
2	デモ用初期データ	WI-004	003_DB の日付変更・追加したデモ用製造指示（計 124 件）への参照	2026-09-22	Claude（ThanhTN の依頼による）
3	デモ用初期データ	WI-005	初期データの備考を日本語化。002_DB に改名（RFC 0010）。英語版・日本語版 PDF を追加（RFC 0008）	2026-09-23	Claude（ThanhTN の依頼による）
4	全体	—	ファイル名に画面の日本語名を使用（RFC 0011）	2026-09-24	Claude（ThanhTN の依頼による）
5	全体	—	テーブル定義テンプレートに合わせて再構成：文書管理、ライフサイクル付きテーブル一覧、ソート順と作成方法付きのインデックス一覧、トリガー一覧、データベースロールの節、マイグレーション表。002_DD を作成済みに更新。初期データのロールバックをマイグレーションに合わせて訂正（ID 接頭辞で削除し、カウンターは残す）	2026-09-28	Claude（ThanhTN の依頼による）

テーブル一覧

テーブルの作成・削除はない。画面 B が扱うテーブル（すべて 001_DB で定義）：

No	スキーマ	物理名	論理名	概要	PK 以外のインデックス	トリガー	作成日	スキーマ変更日	削除日	利用状況
1	public	products	製品（001_DB）	製品参照データ。002_DB での変更：なし — 読み取りのみ。製品列とフィルターの選択肢のために結合する	有	無	2026-09-18	—	—	使用中
2	public	production_orders	製造指示（001_DB）	製造指示 1 件につき 1 行。002_DB での変更：インデックス 2 つの追加、デモ用の初期データ行	有	無	2026-09-18	—（002_DB はインデックスの追加のみ）	—	使用中
3	public	production_order_number_counters	製造指示番号採番（001_DB）	年ごとの製造指示番号の連番。002_DB での変更：投入した製造指示と整合させるための初期データ行のみ	無	無	2026-09-18	—	—	使用中

ER 図とリレーション

001_DB から変更なし — その ER 図を参照。画面 B がたどるリレーションは 1 つだけ：

テーブル	関連テーブル	リレーション（1:1 / 1:N / N:N）	FK 列
products	production_orders	1:N	production_orders.product_id

テーブル定義

列の追加・削除・型変更はない。画面 B が読む列：

production_orders（SCR-002 の読み取り射影）

項目名	物理名	用途
Id	id	行のリンク先 /production-orders/{id}（REQ-025）
OrderNumber	order_number	表示列、ソートキー、部分一致フィルター、ページングのタイブレーカー（REQ-021～REQ-024）
ProductId	product_id	製品フィルター。表示する製品の結合キー（REQ-021、REQ-022）
Quantity	quantity	表示列、ソートキー
DueDate	due_date	表示列、既定のソートキー、範囲フィルター、納期超過マーカ（REQ-021～REQ-023）
Status	status	表示列、ソートキー、複数選択フィルター（REQ-021～REQ-023）
UpdatedAtUtc	updated_at_utc	表示列、ソートキー

notes、created_at_utc、order_year、order_seq、xmin は一覧クエリで**選択しない** — 射影を明示しているので、画面に表示する以上の幅の行を取得することはない。
products.sku と products.name は製品列のために結合を通して選択する（002_BD M-06）。

アクセスパターン

一覧画面は 1 リクエストにつきちょうど 2 つの文を発行する（FN-010）。どちらも EF Core がパラメーター化クエリとして組み立て、適用されたフィルターだけを組み込む — 使われないフィルターは、プランを損なう (@p IS NULL OR ...) 句ではなく、述語そのものを持たない。

ページクエリ

```
SELECT o.id, o.order_number, o.quantity, o.due_date, o.status, o.updated_at_utc,
       p.sku, p.name
FROM production_orders o
JOIN products p ON p.id = o.product_id
WHERE o.status = ANY (@statuses)           -- ステータスフィルターが設定されたときのみ
      AND o.product_id = @productId        -- 製品フィルターが設定されたときのみ
      AND o.due_date >= @dueFrom           -- 設定されたときのみ（両端を含む）
      AND o.due_date <= @dueTo             -- 設定されたときのみ（両端を含む）
      AND o.order_number LIKE @pattern ESCAPE '\' -- 部分文字列が設定されたときのみ
ORDER BY <sort expression> <asc|desc>, o.order_number ASC
LIMIT @pageSize OFFSET (@page - 1) * @pageSize;
```

件数クエリ — 同じ WHERE、SELECT count(*)、そして結合なし：フィルター述語はすべて production_orders にあるので、製品の結合は表示と製品のソートキーにだけ必要である。002_BD M-09 の total とページ数を提供する。

アクセスパターン	頻度	対応するもの
最初のページ、フィルターなし、既定のソート（画面を開いたとき。圧倒的に多いリクエスト）	非常に高い	ix_production_orders_due_date_order_number
納期範囲 ± 既定のソート	高い	同じインデックス（ソート順での範囲スキャン）
製品フィルター ± 既定のソート	中	ix_production_orders_product_id（001_DB、DEC-029）の後にソート
製造指示番号の部分一致	中	ix_production_orders_order_number_trgm
ステータスフィルター	中	専用インデックスなし — 「意図的に追加しなかったインデックス」を参照
数量・ステータス・更新日時・製品でのソート	低い	絞り込んだ集合に対するソートノード
一致件数	ページリクエストごとに 1 回	上と同じ述語

ソートキーの対応

API の sort 値	SQL のソート式	備考
dueDate （既定）	o.due_date	既定の方向は asc （DEC-002）
orderNumber	o.order_number	形式が固定長（ P0-YYYY-NNNN ）なので、辞書順は年内の時系列順と一致する
product	p.sku, p.name	表示ラベルの順序と一致（002_BD M-06）
quantity	o.quantity	
status	CASE o.status WHEN 'Draft' THEN 1 WHEN 'InProgress' THEN 2 WHEN 'Completed' THEN 3 WHEN 'Cancelled' THEN 4 END	アルファベット順ではなくワークフロー順 — アルファベット順では「Cancelled、Completed、Draft、In progress」となり、計画担当者には意味がない
updatedAt	o.updated_at_utc	

o.order_number ASC をすべてのソートに付け加える（ sort=orderNumber のときは唯一のキー）。一意なので全体の順序が決定的になり、ページングでページ間に行が重複したり抜けたりすることはない（REQ-023）。

クエリに届くのは、アプリケーションの許可リスト（002_BD V-13）を通ったこの 6 つの値だけである。クライアントの文字列を SQL に埋め込むことはない（ ai/rules/database.md ）。

インデックス一覧

001_DB の既存インデックスは変更しない。002_DB で新規（どちらも稼働中の production_orders テーブルに AddProductionOrderListIndexes が作成）：

No	テーブル	インデックス名	方式	一意	列とソート順	部分インデックス条件 / INCLUDE	CONCURRENTLY で作成	作成日	削除日	根拠（アクセスパターン）
1	production_orders	ix_production_orders_due_date_order_number	btree	no	due_date ASC, order_number ASC	—	yes — 稼働中のテーブル	2026-09-22	—	既定のソートと納期範囲（注 1）
2	production_orders	ix_production_orders_order_number_trgm	gin	no	order_number (gin_trgm_ops)	—	yes — 稼働中のテーブル	2026-09-22	—	製造指示番号の部分一致検索（注 2）

1. 画面の既定で最も頻繁なクエリ：製造指示番号をタイブレーカーとして納期順に並べ、任意で納期範囲に絞る。順序付けと範囲述語の両方を 1 つのインデックスで賄うので、通常のページにソートノードは不要。

2. 製造指示番号の部分一致フィルターは含む一致（REQ-022）であり、btree では対応できない。トライグラムインデックス（`pg_trgm` の演算子クラス `gin_trgm_ops`）で `LIKE` `'%...%'` をインデックスで処理できるようにする（DEC-010）。

`ix_production_orders_order_number_trgm` には `pg_trgm` 拡張が必要で、同じマイグレーションで作成する。`pg_trgm` は PostgreSQL 17 では信頼済み拡張なので、マイグレーションを実行するオーナーのログインはスーパーユーザー権限なしで作成でき、実行時ログイン（`pmai_app`）はこれを使うための新しい権限を必要としない。

トライグラムインデックスに関する注意：

- アプリケーションは部分文字列を大文字にして、`ILIKE` ではなく `LIKE` で照合する：`order_number` は生成列で常に大文字（`PO-YYYY-NNNNN`）なので、入力を大文字にするだけで大文字小文字を区別しない照合になり、通常の `LIKE` なら `upper(order_number)` の関数インデックスなしでインデックスを使える。
- 3 文字未満の部分文字列は完全なトライグラムを生まないため、PostgreSQL は絞り込んだ集合のスキャンにフォールバックする。これは許容する：002_BD V-09 は部分文字列を 20 文字までに制限しており、1～2 文字の検索はまれで、テーブルはフォールバックが安価に済む大きさである。問題になれば、UI で検索前に 3 文字を要求できる。

意図的に追加しなかったインデックス

候補	追加しない理由
<code>status</code> （単独、または <code>(status, due_date)</code> ）	値は 4 つだけで、よく使う 2 つ（ <code>Draft</code> 、 <code>InProgress</code> ）が大半の行を占めるため、ステータスのインデックスは既に絞り込まれた結果のスキャンに勝るほどの選択性がない。複 合 <code>(status, due_date)</code> なら「ステータスフィルター + 既定のソート」に対応できるが、 <code>status = ANY(...)</code> （複数選択、DEC-005）では PostgreSQL が選択された値をま たいでマージまたは再ソートしなければならない。ステータスで絞った一覧が主要なアクセスパターンになった場合、または <code>production_orders</code> が約 100 万行を超えた場合に見直 す
<code>(updated_at_utc, order_number)</code> 、 <code>(quantity, ...)</code> 、 <code>(status_rank, ...)</code>	ソートキーごとにインデックスを作ると 5 つ増え、画面で最も使われない並び順のために挿入・更新のたびに遅くなる。これらのソートは絞り込んだ集合に対するソートノードとして実行さ れ、想定件数では十分予算内である（性能見込みを参照）
<code>(product_id, due_date, order_number)</code>	「製品フィルター + 既定のソート」を 1 回のスキャンで賄えるが、001_DB が残した FK インデックス（DEC-029）を置き換え、EF Core の規約が管理するインデックスを変えること になる。製品フィルターだけで集合はテーブルの約 1/30 に減り、その後のソートは軽い。001_DB のインデックス構成を変えるほどの価値はない

トリガー一覧

なし — トリガーはない。

制約

制約の追加・削除・変更はない。001_DB のすべての制約は定義どおりのままであり、001_DB がアプリケーションで強制するとしたルール（納期 ≥ 今日、ステータス遷移、製品・数量のロック）も含む。画面 B は読み取り専用（DEC-003）なので、いずれにも違反しない。

納期超過マーカー（002_BD M-08、FN-013）はデータベースの概念ではない： `due_date`、`status`、工場現地の現在日付から行ごとに計算する。格納する列でもフィルターでもない — データベースは `due_date` と `status` を返し、アプリケーションが判定する。001_DB が納期ルールについて挙げたのと同じ理由で「今日」をスキーマに持ち込まない：格納・生成した超過フラグは、日付が変わった瞬間に誤りになる。

トランザクションと同時実行制御

操作	トランザクション範囲	同時実行制御
一覧のページエリ + 件数（FN-010）	明示的なトランザクションなし。1 リクエスト内の 2 つの文を、それぞれ <code>READ COMMITTED</code> で実行	不要。2 つの文は異なるスナップショットを見ることがあるので、その間に作成された行によって合計件数がページと 1 件ずれることがある。これはページング一覧では普通のことであり、REPEATABLE READ のトランザクションで直すのではなく許容する：画面は情報提供用で、次のクエリで正され、合計件数が行とトランザクション上一致していることに依存するものはない

画面 B はロックを取らず、何も書き込まない。

デモ用初期データ

WI-004（003_DB）以降: 投入済みの `Completed` の製造指示 16 件は日付を変更し、さらにデモ用製造指示を 44 件追加した。そのためデモ用データベースは 124 件（`Draft` 35、`InProgress` 25、`Completed` 56、`Cancelled` 8）を持ち、カウンターはその後に続く。下表は 002_DB 本来の 80 行を説明する。残りは 003_DB の「デモ用初期データ」を参照。

DEC-007（ユーザー）は、ページング・ソート・フィルター・納期超過マーカーを実演でき、E2E テストのフィクスチャになるよう、製造指示の初期データを要求している。001_DB が既に 30 製品を投入しており、002_DB はそれらにまたがって**製造指示 80 件**を投入する。

性質	値
行数	80（ユーザーが求めた 60～100 の範囲内。既定のページサイズ 20 で 4 ページ、50 でも 1 ページを超える）
ステータスの分布	<code>Draft</code> 32、 <code>InProgress</code> 24、 <code>Completed</code> 16、 <code>Cancelled</code> 8 — どのフィルターの組み合わせでも、空でなく大きさの異なる結果になる
製品	投入済みの 30 製品すべてに分散し、複数の製造指示を持つ製品もあるので、製品フィルターと製品ソートの両方に意味がある

性質	値
数量	1 から約 5,000。境界値 1 と大きな値を含むので、数量ソートの結果が目に見えて意味を持つ
納期	マイグレーション実行時の工場現地日付に対する相対値（DEC-011）：過去 20 件（1～14 日前）、今日 2 件、以後 40 日間に 58 件。80 件のうち 25 件は意図的に他と同じ納期を持たせ、製造指示番号のタイプレーカーが実際に働くようにした。投入結果の納期は 55 通り
納期超過の行	納期を過ぎた <code>Draft</code> と <code>InProgress</code> の行 — 過去 20 件のうち 15 件 — に納期超過マーカが付く。残り 5 件は <code>Completed / Cancelled</code> で、意図的にマーカが付かず、M-08 の後半を実演する
備考	ある行とない行がある（一覧は備考を表示しない。画面 A のために投入行を現実的に保つ）。 <code>Demo order N</code> として投入し、WI-005 以降はマイグレーション <code>LocalizeDemoDataToJapanese</code> （001_DB）が デモ用の製造指示 N に書き換える
製造指示番号	PO-<投入年>-00001 ～ PO-<投入年>-00080 として発行し、その年の <code>production_order_number_counters</code> を <code>last_seq = 80</code> で投入するので、画面 A で次に作成する製造指示は衝突せずに連番を続ける。API で最初に作成した製造指示が <code>00081</code> であることを統合テストで確認する
タイムスタンプ	<code>created_at_utc</code> / <code>updated_at_utc</code> を実行日の前の数週間にずらして分散させるので、「更新日時」列とそのソートがすべて同じにはならない

この初期データの 2 つの性質は意図的なものであり、明確に述べておく：

- 1. **納期は固定の暦日ではなく実行日に対する相対値である**（DEC-011）。固定日付のマイグレーションのほうが再現性は高いが、数か月後には投入済みの製造指示がすべて納期超過になり、画面はその後ずっと見栄えの悪いデモになる。相対日付なら、スタックを作り直すたびに現実的な組み合わせが保たれる。その他の初期データ — ID、製造指示番号、製品、数量、ステータス、日数のオフセット自体 — はすべて固定なので、データは実行日が決まれば決定的であり、テストは絶対日付ではなく件数・ステータス・相対日付を検証する。
- 2. **初期データは空のテーブルにのみ投入される**。挿入は `WHERE NOT EXISTS (SELECT 1 FROM production_orders)` で保護されているので、既に実際の製造指示を持つデータベースにマイグレーションを適用しても何も追加されない。したがって再実行・再適用は幂等である。

日付を実行時に計算するため、この初期データには EF Core `HasData`（001_DB の製品の初期データのように定数値を要求する）を使えない。生成した連番に対する `INSERT ... SELECT` を明示的な `migrationBuilder.Sql(...)` として記述しており、これもパラメーターなしで決定的である。固定の行 ID は、`ProductSeed` と同じ形式の UUIDv7 形の定数である。

データベースロールと権限

変更なし。pg_trgm の演算子クラスを使うのに権限付与は不要。

ロール	使用者	これらのテーブルへの権限	理由
<code>pmai_app</code> （実行時ロギン）	実行時の API	<code>production_orders</code> と <code>products</code> に <code>SELECT</code> — 001_DB で付与済み（DEC-016）	一覧クエリと件数クエリは読み取りのみ。画面 B は <code>production_order_number_counters</code> への権限を必要とせず、付与もしない

ロール	使用者	これらのテーブルへの権限	理由
オーナー (<code>POSTGRES_USER</code>)	マイグレーションのみ	オーナー — DDL	すべてのマイグレーションと同様に、拡張・インデックス・初期データを作成する (<code>deploy/README.md</code>)

API / DD 対応

DD 項目名は 002_BD \$3 の変数名。API 項目名は 002_DD-API で確定している。一覧のレスポンスは、列を持たない計算済みのレスポンス専用項目を 1 つ追加する。

テーブル.列	DD 項目	API 項目
<code>production_orders.id</code>	(行のリンク先)	<code>id</code>
<code>production_orders.order_number</code>	行の製造指示番号 / <code>orderNumber</code> フィルター	<code>orderNumber</code> (行)、 <code>orderNumber</code> (クエリ : 部分文字列)
<code>production_orders.product_id</code>	<code>productId</code> フィルター	<code>productId</code> (クエリ)
<code>products.sku</code> 、 <code>products.name</code>	行の製品ラベル (M-06)	<code>product.sku</code> 、 <code>product.name</code>
<code>production_orders.quantity</code>	行の数量	<code>quantity</code>
<code>production_orders.due_date</code>	行の納期、 <code>dueFrom</code> / <code>dueTo</code> フィルター	<code>dueDate</code> (行、ISO YYYY-MM-DD)、 <code>dueFrom</code> 、 <code>dueTo</code> (クエリ)
<code>production_orders.status</code>	行のステータス、 <code>status</code> フィルター	<code>status</code> (行)、 <code>status</code> (クエリ、繰り返し可)
<code>production_orders.updated_at_utc</code>	行の最終更新	<code>updatedAt</code> (ISO 8601 UTC)
— (<code>due_date</code> 、 <code>status</code> 、工場の今日から計算)	行の納期超過マーカー (M-08)	<code>isOverdue</code> (レスポンスのみ)
— (件数クエリで計算)	結果の概要 (M-09)	<code>total</code> (レスポンスのみ)
— (クエリ制御値をそのまま返す)	ソート、方向、ページ、ページサイズ	<code>sort</code> 、 <code>dir</code> 、 <code>page</code> 、 <code>pageSize</code>
<code>production_orders.notes</code> 、 <code>created_at_utc</code> 、 <code>order_year</code> 、 <code>order_seq</code> 、 <code>xmin</code>	—	一覧エンドポイントでは公開しない

性能見込み

インデックスの選択を好みの問題ではなく検証可能にするための規模の前提：1 工場が 1 日に約 50 件の製造指示を作成すると年間約 15,000 行となり、テーブルは何年も数十万行台にとどまる。デモ用データベースは 80 行。

クエリ	見込み
既定のページ（フィルターなし）	<code>ix_production_orders_due_date_order_number</code> のインデックススキャンで、 <code>pageSize</code> 行とオフセット分で停止。ソートノードなし
納期範囲 + 既定のソート	同じインデックスの範囲限定スキャン
製造指示番号の部分一致	トライグラムインデックスのビットマップインデックススキャンの後にフィルター
製品フィルター	<code>ix_production_orders_product_id</code> のインデックススキャンの後、テーブルの約 1/30 のソート
ステータスフィルター、または既定以外のソートキー	絞り込んだ集合のスキャンとソートノード — 上記の規模なら許容範囲
件数	同じ述語。画面は正確な合計を報告するので（REQ-024）、推定値ではなく正確な <code>count(*)</code>

テーブルが約 100 万行を超えた場合、深いページング（大きな `pageSize` での大きな `page`）が一般的になった場合 — `OFFSET` は線形に伸びるので、画面が規定する「n / m ページ」操作を犠牲にしてキーセットページングが望ましくなる — またはステータスフィルターが主要なアクセスパターンになった場合に、この設計を見直す。

マイグレーションの影響と復旧限界

マイグレーション	種別	対象テーブル	稼働中のテーブルか	手順
<code>AddProductionOrderListIndexes</code> （2026-09-22）	追加	<code>production_orders</code> （インデックス 2 つ）、 <code>pg_trgm</code> 拡張	yes	単一ステップ。インデックスは <code>CONCURRENTLY</code> で作成
<code>SeedDemoProductionOrders</code> （2026-09-22）	データ挿入（保護付き）	<code>production_orders</code> 、 <code>production_order_number_counters</code>	yes	単一ステップ

2 つのマイグレーションを次の順に実行する：

1. `AddProductionOrderListIndexes` — インデックスと拡張。
- **マイグレーション種別:** 追加。 `CREATE EXTENSION IF NOT EXISTS pg_trgm` の後、両方のインデックス。
 - **同時実行:** `production_orders` は既に使用中なので、`ai/rules/database.md` に従い両方のインデックスを `CREATE INDEX CONCURRENTLY IF NOT EXISTS` で作成する。
`CONCURRENTLY` はトランザクション内では実行できず、EF Core はマイグレーションをトランザクションで包むため、これらの文は `migrationBuilder.Sql(..., suppressTransaction: true)` で発行する。その結果を実行前に理解しておく必要がある：このマイグレーションは**アトミックではない**。途中で失敗すると PostgreSQL は `INVALID` なインデックスを残す。これはどのクエリにも使われず、再試行の前に削除（`DROP INDEX CONCURRENTLY`）しなければならない。`SELECT indexrelid::regclass FROM pg_index WHERE NOT indisvalid;` で確認する。
 - **データ復旧限界:** なし — データの書き込み・変更はない。

- **ロールバック計画:** 両方のインデックスに `DROP INDEX CONCURRENTLY IF EXISTS`。拡張はそのまま残す（削除は不要であり、拡張を使う他のオブジェクトを壊しうる）。このマイグレーションを戻しても一覧が遅くなるだけで、誤りにはならない。

2. `SeedDemoProductionOrders` — デモ用データ。

- **マイグレーション種別:** データ挿入（保護付き）。`production_orders` が空のときだけ、`production_orders` に 80 行、`production_order_number_counters` に 1 行を挿入する。
- **データ復旧限界:** 初期データは空のテーブルにしか書き込まないので、適用して失われるものはない。戻すと投入済みの 80 行を**固定 ID** で削除するため、ユーザーが画面 A で投入済みの製造指示に加えた編集も削除される — その編集はマイグレーションからは復元できない。ユーザーが作成した製造指示は、その ID が投入集合に含まれないので影響を受けない。
- **ロールバック計画:** `DELETE FROM production_orders WHERE id::text LIKE '0197e4a0-0000-7000-8001-%'` で、投入済みの 80 行を固定 ID の接頭辞で削除する。カウンターはそのまま残すので、番号は前に進み続け、製造指示番号が再利用されることはない。初期データの後に作成された行は影響を受けない。
- **運用上の注意:** 納期は実行日から計算されるので、後日新しいボリュームでこのマイグレーションを再適用すると異なる納期になる（設計どおり、DEC-011）。したがってテストの検証は、件数・ステータス・今日からのオフセットに対して書き、絶対日付に対しては決して書かない。

どちらのマイグレーションもオーナーのログインで実行し、`pmai_app` の権限の変更は不要である。

どちらも 2026-09-22 にローカルの Compose データベースに適用した：2 つのインデックスが存在し、`pg_index` は `INVALID` なインデックスを報告せず（つまりアトミックでない `CONCURRENTLY` の経路は完了した）、テーブルは 55 通りの納期にまたがる投入済みの製造指示 80 件を持つ。

初期データは統合テスト用データベースにも入る。 このデータベースは使い捨ての PostgreSQL コンテナに対して同じマイグレーションを実行する。これは意図的である — 一覧のテストには現実的なデータが必要で、実際のマイグレーション一式をテストすることが目的である — が、そこでは製造指示番号が投入済みの 80 件の後から始まることを意味する。その影響は WI-003 DEC-012 に記録している。

未決事項

決定事項	選択肢	推奨	状態
大文字小文字を区別しない製造指示番号の部分一致検索のインデックス	<code>pg_trgm</code> の GIN インデックス／前方一致のみの <code>upper(order_number)</code> の btree／インデックスなし	<code>pg_trgm</code> GIN — REQ-022 の「含む」の意味をインデックスで保てる唯一の選択肢	決定済み — <code>work-items/WI-003/decisions.md</code> DEC-010
初期データの納期：固定の暦日か、マイグレーション実行日に対する相対値か	固定（ <code>HasData</code> 、完全に再現可能）／相対（ <code>INSERT ... SELECT</code> 、いつまでも現実的）	相対。その他はすべて固定し、挿入は空のテーブルに限定	決定済み — DEC-011
ステータスフィルター専用のインデックス	<code>(status, due_date)</code> ／なし	当面なし。約 100 万行を超えるか、ステータスでの絞り込みが主になったら見直す	決定済み — ここに記録。別途 DEC は不要

決定事項	選択肢	推奨	状態
OFFSET ではなくキーセットページング	キーセット（深いページが速い、ページ番号なし）／オフセット（ページ番号あり、線形コスト）	オフセット — 画面は「n / m ページ」を規定しており（REQ-024）、件数はキーセットを正当化しない	決定済み — ここに記録